Data Handling: Import, Cleaning and Visualisation

Notes for Exercise/Workshop 1: Tools, Working with Text Files

Dr. Aurélien Sallin

01/10/2023

Prerequisites

- Sign in to Nuvolos here: https://az.nuvolos.cloud/login using your Unisg E-Mail address.
- Nuvolos is a cloud-based service that allows you to use R and the IDE RStudio through your browser. In the scope of this course, there is no need to install R locally on your machine.
- If you do not have a GitHub account, you can create one here (not required for exam): https://github.com/join. Accounts are free. No need to use your Unisg E-Mail address.

Part I: Introduction

Why R?[1]

The 'data language'

The programming language and open-source statistical computing environment R has over the last decade become a core tool for data science in industry and academia. It was originally designed as a tool for statistical analysis. Many characteristics of the language make R particularly useful to work with data. With the rise of the 'data economy' and 'data science', R is increasingly used in various domains, going well beyond the traditional applications of academic research.

High-level language, relatively easy to learn

R is a relatively easy computer language to learn for people with no previous programming experience. The syntax is rather intuitive and error messages not too cryptic to understand (facilitates learning by doing). Moreover, with R's recent stark rise in popularity, there are plenty of freely accessible resources online that help beginners to learn the language.

Free, open-source, large community

Due to its vast base of contributors, R serves as a valuable tool for users in various fields related to data analysis and computation (economics/econometrics, biomedicine, business analytics, etc.). R users have direct access to thousands of freely available 'R-packages' (small software libraries written in R), covering diverse aspects of data analysis, statistics, data preparation, and data import.

Hence, a lot of people using R as a tool in their daily work do not actually 'write programs' (in the traditional sense of the word) but apply those packages. Applied econometrics with R is a good example of this. Almost any function a modern commercial computing environment with a focus on statistics and econometrics (such as STATA) is offering can also be found within the R environment. Furthermore, there are R packages covering all the areas of modern data analytics, including natural language processing, machine learning, big

[1] From Matter (2018)

data analytics, etc. (see the CRAN Task Views for an overview). We thus do not actually have to write a program for many tasks we perform with R. Instead, we can build on already existing and reliable packages.

The tools: R/RStudio

R is the high-level (meaning 'more user friendly') programming language for statistical computing. Once we have installed R on our computer, we can run it...

a. ... locally:

- Install the latest R version. R is open-source and freely available from here: https://stat.ethz.ch/CRAN/. Click on the respective link for your operating system (Windows, Mac, Linux), and follow the instructions.
- Once R is installed, install RStudio from here: https://www.rstudio.com/products/rstudio/download/#download. Under "Installers for Supported Platforms", click on the respective installer for your operating system (Windows, Mac, etc.) and follow the instructions.

c. ... using Nuvolos:

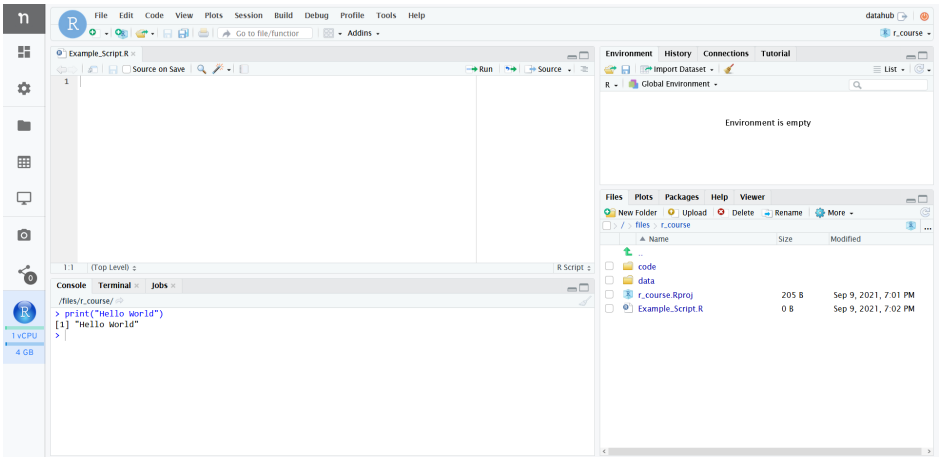

Figure 1: Running R in RStudio (IDE).

The latter is what we will do throughout this course.

In the following, we have a look at each of the main panels we will use in this course.

The R-Console

When working in an interactive session, we simply type R commands directly into the R console. Typically, the output R returns, when we give it a command this way, is printed in the console as well.

For example, we can tell R to print the phrase `Hello world` to the console by typing to following command in the console and hitting enter:

```
print("Hello world")
```

```
## [1] "Hello world"
```

R-Scripts

Apart from very short interactive sessions, it usually makes sense to write R code not directly in the command line but to a R-script in the script panel. This way, execute several lines at once, comment the code (to explain what it does), save it on our hard disk, and further develop the code later on.

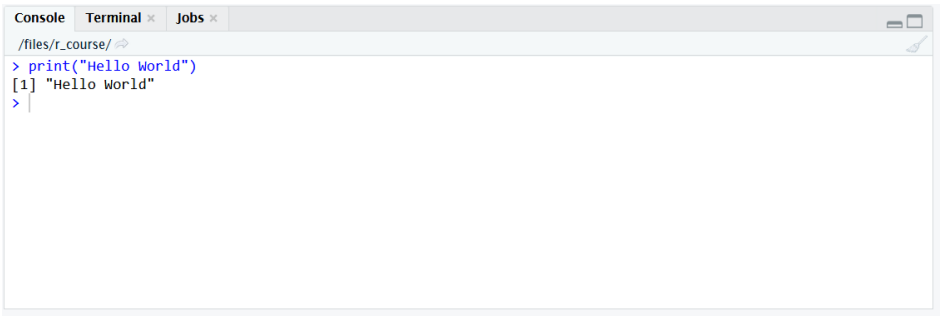

Figure 2: Running R in the Mac/OSX terminal.

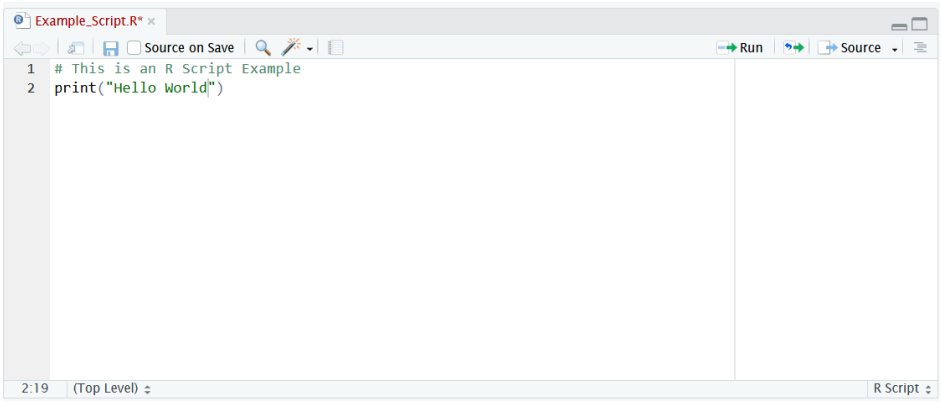

Figure 3: The R Script window in RStudio.

R Environment

The environment pane shows what variables, objects, and data are loaded in our current R session. Moreover, it offers functions to open documents and import data.

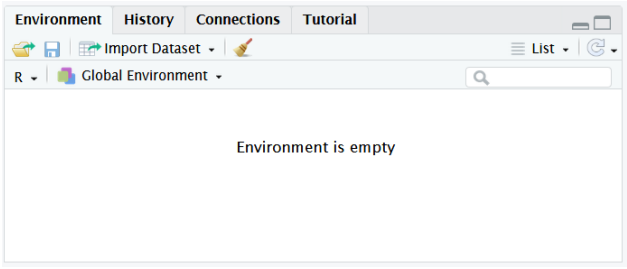

Figure 4: The environment window in RStudio.

File Browser

With the File browser window, we can navigate through the folder structure and files on our computer's hard disk, modify files, and set the working directory of our current R session. Moreover, it has a pane to show plots generated in R and a pane with help pages and R documentation.

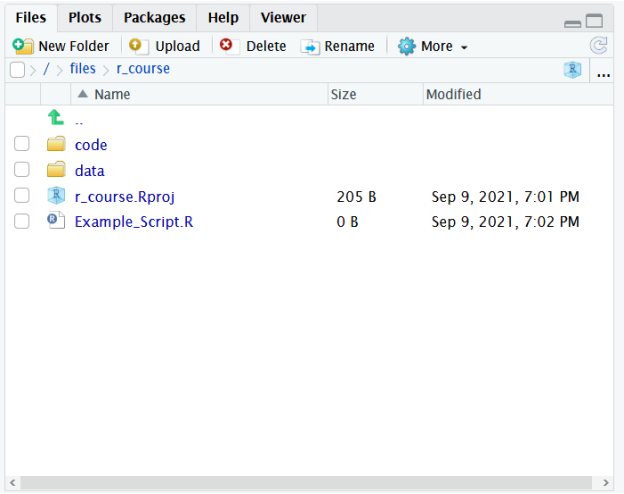

Figure 5: The file browser window in RStudio.

A Note on GitHub

What is GitHub?

GitHub is a code hosting platform where developers can store and manage their code. GitHub allows for version control and collaboration (it builds on the version control system Git). On GitHub, you and others can work together on projects from anywhere. Usually, one repository (or repo) is used to organize a single project. Repositories can be public (for everyone to see) or private (only for you and your team). There are functionalities like branching for version control (i.e., making a temporary separate copy of the project) or raising issues.

Version Control: Why make such a Big Deal?

Have you ever stored files like this: 'file_final.txt', 'file_really_final.txt', 'file_really_final2.txt', etc.? Gets out of hands quickly (even more so if you work in a team)! Also, in software development, it is not safe if a developer works on the 'official' source code directly (the definitive code on which a program runs, referred to as the master branch on GitHub). Rather, the developer works on a separate copy of the master branch (referred to as a branch). Once the developers' suggested changes have been approved by others, they can be merged from the separate branch to the master branch.

What are Issues?

Raising an issue means that you create a flag to keep track of bugs, enhancements, or team members' tasks. You can think of issues in a similar fashion as of e-mails. However, issues can be shared and discussed. They are also helpful when using a (publicly accessible) program written by someone else: if you suspect there is a bug in the program, you can go to the GitHub repository associated with the program and raise an issue. Sometimes, you will see that someone else has already reported the bug, and possibly the owner of the repo has already suggested a solution.

Why is GitHub helpful for Economists and Data Scientists?

GitHub is useful in data science for reasons similar to those in software development: whenever you work with code, you will want to check if your changes do not break anything before merging them with the definitive code. Moreover, many data scientists or econometricians publicly share their programs on GitHub. You can thus build on what others have done, interact with them, and upload your own code for others to use and to get feedback on it. Your GitHub repo can also enable others to replicate your research results. As this

course shows, GitHub is convenient to store and work on code, but it can incorporate other components like data, text components, or slides. We encourage you to contribute to this course by raising issues (if you spot mistakes or have ideas for improvements). As a side note: in principle, you can also use GitHub for version control in a completely coding-free project.

Part II: First steps with R

Before introducing some of the key functions and packages for data analysis with R, we should understand how such programs basically work and how we can write them in R. Once we understand the basics of the R language and how to write simple programs, understanding and applying already implemented programs is much easier.[2]

HELP ME!!!

If you are unsure what a particular command does, you can look up its documentation by putting a question mark in front of it (works both with and without parentheses).

```
# This will open the help file/documentation for the respective function
?sum
?names()
```

Variables and Vectors

```
# a simple integer vector
a <- c(10, 22, 33, 22, 40)

# give names to vector elements
names(a) <- c("Andy", "Betty", "Claire", "Daniel", "Eva")
a
```

```
##   Andy  Betty Claire Daniel    Eva
##     10     22     33     22     40
```

```
# indexing either via number of vector element (start count with 1)
# or by element name
a[3]
```

```
## Claire
##     33
```

```
a["Claire"]
```

```
## Claire
##     33
```

```
# Inspect the object you are working with
class(a) # returns the class(es) of the object
```

```
## [1] "numeric"
```

```
str(a) # returns the structure of the object ("what is in variable a?")
```

```
##  Named num [1:5] 10 22 33 22 40
##  - attr(*, "names")= chr [1:5] "Andy" "Betty" "Claire" "Daniel" ...
```

[2] In fact, since R is an open source environment, you can directly look at already implemented programs in order to learn how they work.

Math Operators

R knows all basic math operators and has a variety of functions to handle more advanced mathematical problems. One basic practical application of R in academic life is to use it as a sophisticated (and programmable) calculator.

```
# basic arithmetic
2+2
```

```
## [1] 4
```

```
sum_result <- 2+2
sum_result
```

```
## [1] 4
```

```
sum_result -2
```

```
## [1] 2
```

```
4*5
```

```
## [1] 20
```

```
20/5
```

```
## [1] 4
```

```
# order of operations
2+2*3
```

```
## [1] 8
```

```
(2+2)*3
```

```
## [1] 12
```

```
(5+5)/(2+3)
```

```
## [1] 2
```

```
# work with variables
a <- 20
b <- 10
a/b
```

```
## [1] 2
```

```
# arithmetic with vectors
a <- c(1,4,6)
a * 2
```

```
## [1]  2  8 12
```

```
b <- c(10,40,80)
a * b
```

```
## [1]  10 160 480
```

```
a + b
```

```
## [1] 11 44 86
```

```
# other common math operators and functions
4^2
```

```
## [1] 16
```

```
sqrt(4^2)
```

```
## [1] 4
```

```
log(2)
```

```
## [1] 0.6931472
```

```
exp(10)
```

```
## [1] 22026.47
```

```
log(exp(10))
```

```
## [1] 10
```

```
# special numbers
# Euler's number
exp(1)
```

```
## [1] 2.718282
```

```
# Pi
pi
```

```
## [1] 3.141593
```

References

Matter, Ulrich. 2018. "A Brief Introduction to Programming with r." Lecture notes. St. Gallen: University of St. Gallen.